



**TECNOLÓGICO
NACIONAL DE MÉXICO**

INSTITUTO TECNOLÓGICO SUPERIOR DE JEREZ

Carrera: Ingeniería en Sistemas Computacionales

Nombre de la Materia: Tópicos Selectos Web

**Nombre de la Actividad: Reporte de Practica
(FIREBASE)**

Nombre del Alumno(a): Nayeli Morales García

Número de Control: S19070105

Semestre: 14

Nombre del Docente: Salvador Acevedo Sandoval

Fecha de Entrega: miércoles 20 de mayo del 2026

Introducción

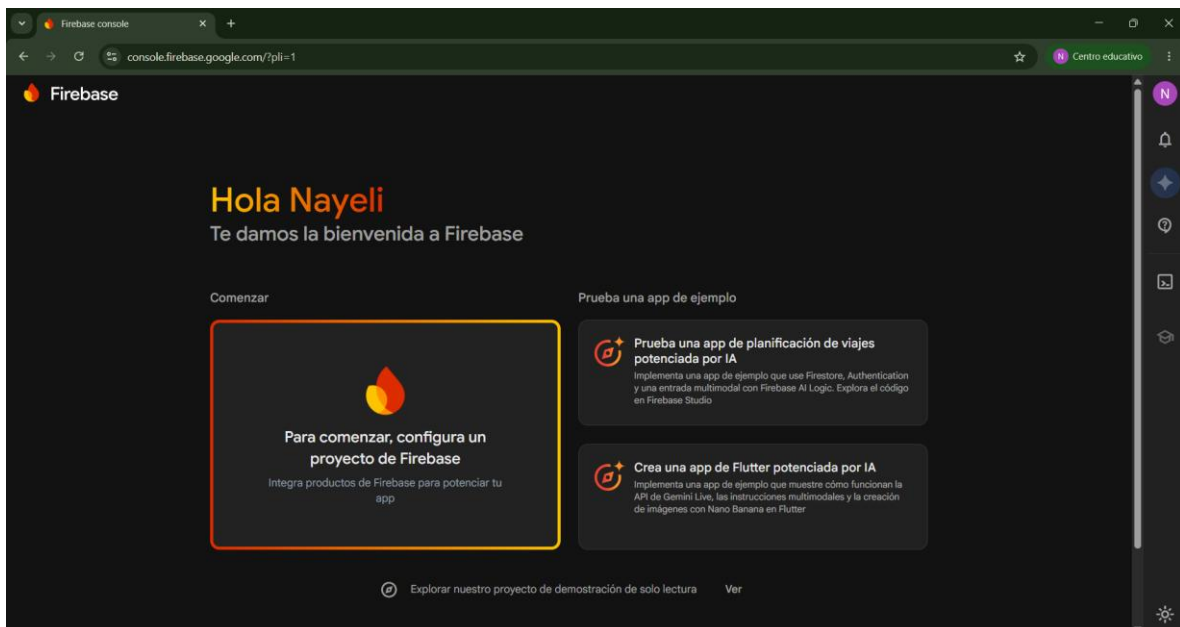
En la actualidad, el desarrollo de aplicaciones móviles y web requiere herramientas que permitan gestionar información en tiempo real, autenticación de usuarios y almacenamiento de datos de manera eficiente. Firebase es una plataforma desarrollada por Google que proporciona diversos servicios en la nube para facilitar el desarrollo de aplicaciones modernas.

La presente práctica tiene como objetivo conocer el funcionamiento y las principales herramientas que ofrece Firebase, así como su implementación en aplicaciones para mejorar la administración de bases de datos, autenticación y almacenamiento en tiempo real. Durante el desarrollo de la práctica se realizaron configuraciones básicas de la plataforma, conexión con la aplicación y pruebas de funcionamiento de sus servicios.

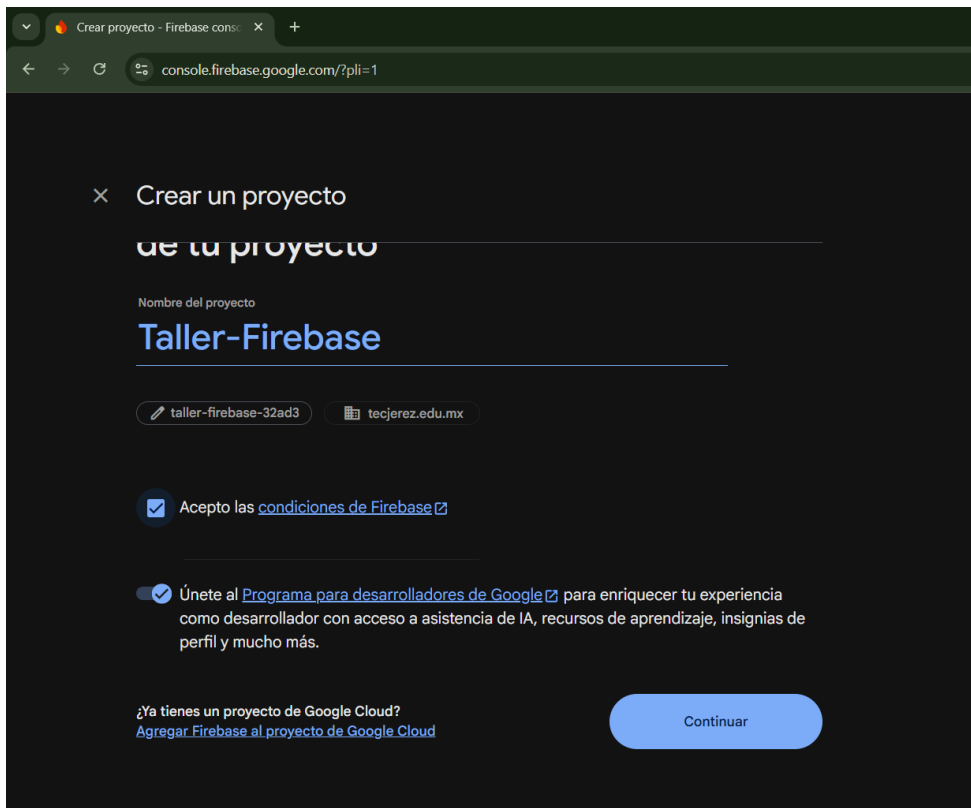
El uso de Firebase permite simplificar procesos de desarrollo, reducir tiempos de implementación y mejorar la experiencia del usuario mediante servicios integrados y fáciles de utilizar. Por ello, esta práctica contribuye al aprendizaje de tecnologías actuales ampliamente utilizadas en el ámbito del desarrollo de software.

Desarrollo

Primero ingresamos a la página de Firebase y aparecerá la pantalla así. Le damos en el botón que aparece en grande que es para comenzar o crear un proyecto para nuestro taller.

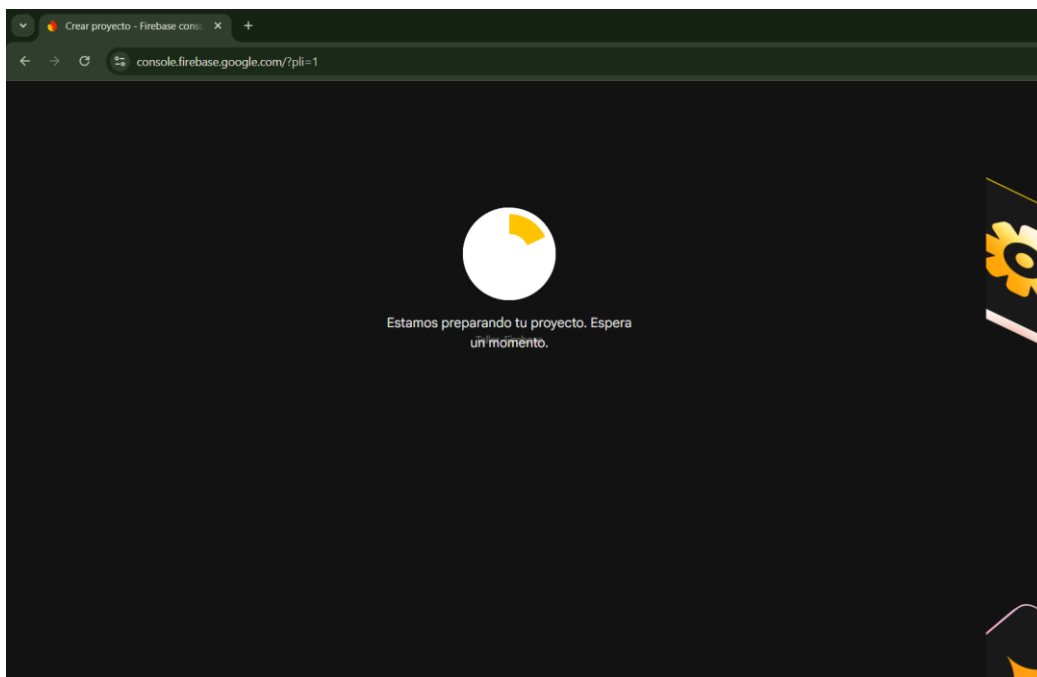


Luego nos manda a esta página, agregamos el nombre del proyecto aceptamos las condiciones y le damos en seleccionar el recurso y en aceptar.

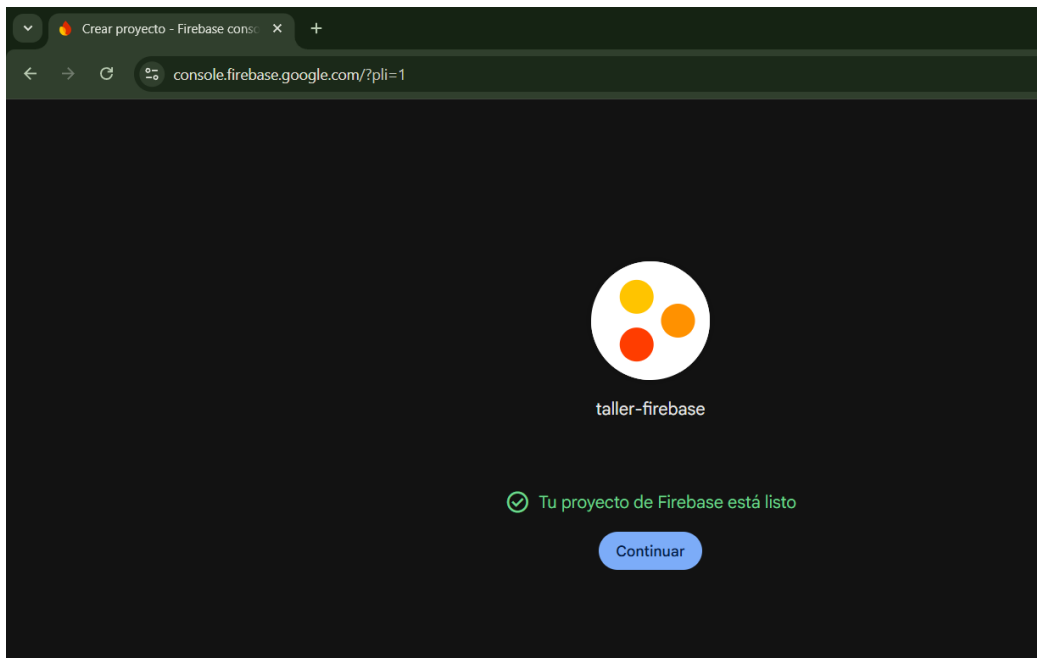


The screenshot shows the 'Crear un proyecto' (Create a project) page in the Firebase console. The page has a dark theme. At the top, there's a title 'Crear un proyecto' with a close button. Below it, the subtitle 'de tu proyecto' is partially visible. The main section is titled 'Nombre del proyecto' (Project name) and contains a text input field with the value 'Taller-Firebase'. Below the input field, there are two buttons: 'taller-firebase-32ad3' and 'tecjerez.edu.mx'. Further down, there are two checkboxes. The first is checked and labeled 'Acepto las condiciones de Firebase' (I accept the Firebase conditions). The second is also checked and labeled 'Únete al Programa para desarrolladores de Google' (Join the Google Developer Program), with a description: 'para enriquecer tu experiencia como desarrollador con acceso a asistencia de IA, recursos de aprendizaje, insignias de perfil y mucho más.' (to enrich your experience as a developer with access to AI assistance, learning resources, profile badges and much more). At the bottom left, there's a link '¿Ya tienes un proyecto de Google Cloud? Agregar Firebase al proyecto de Google Cloud' (Do you already have a Google Cloud project? Add Firebase to the Google Cloud project). At the bottom right, there's a blue 'Continuar' (Continue) button.

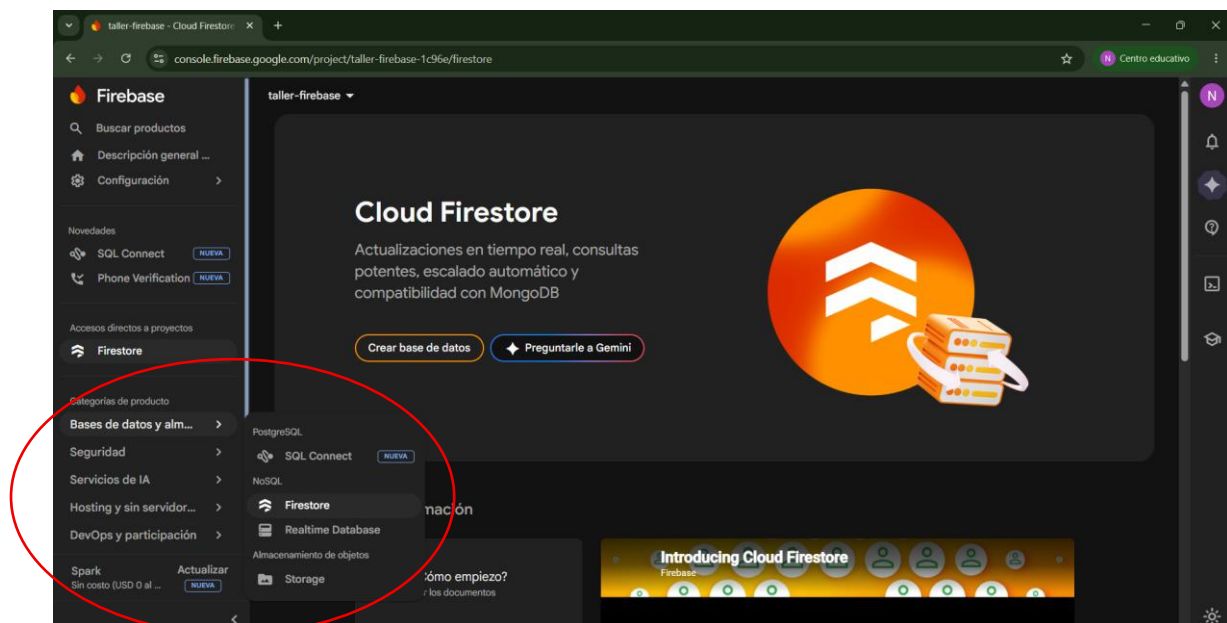
Ahora esperamos a que se cree el proyecto, que ya se está cargando.



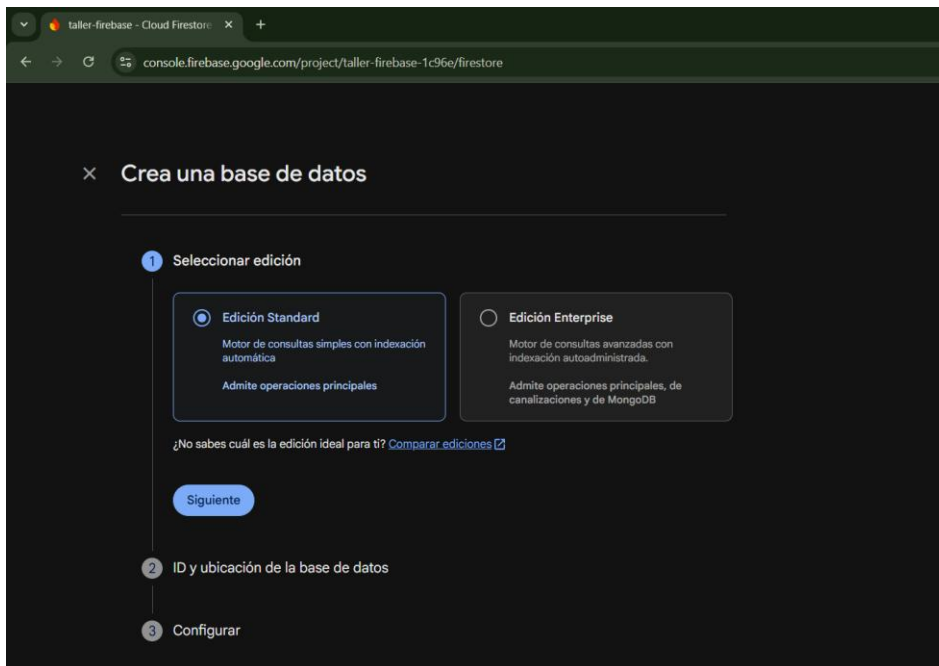
Ya cuando aparezca, así como en la imagen es que ya está listo y creado, le damos en continuar.



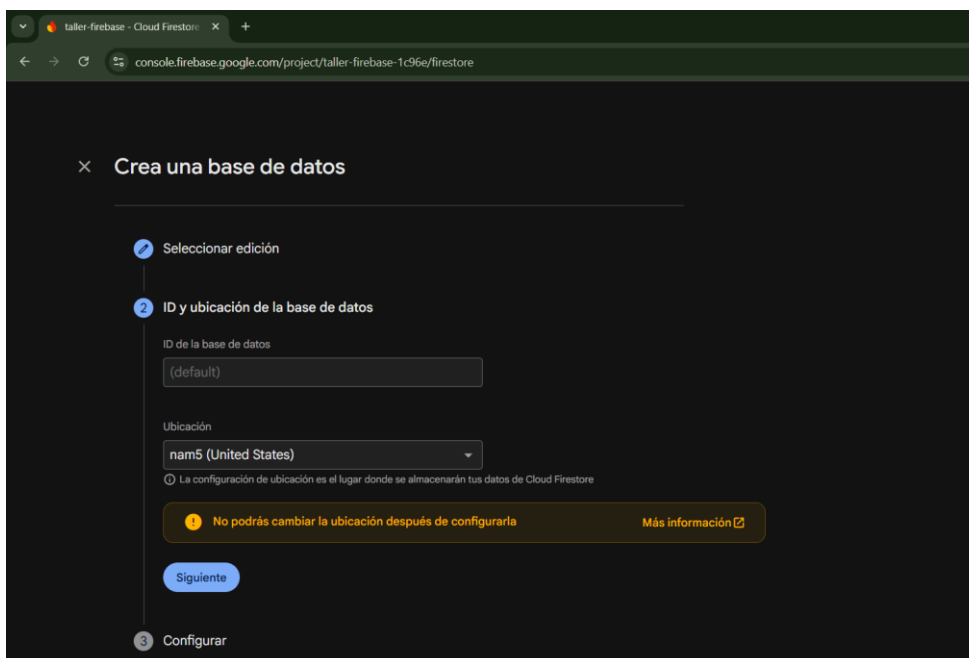
Nos aparecerá así, ahora para crear la base de datos vamos al menú lateral izquierdo en la opción de base de datos colocamos el mouse y nos aparecerá una pequeña ventana y le damos click en Firebase.



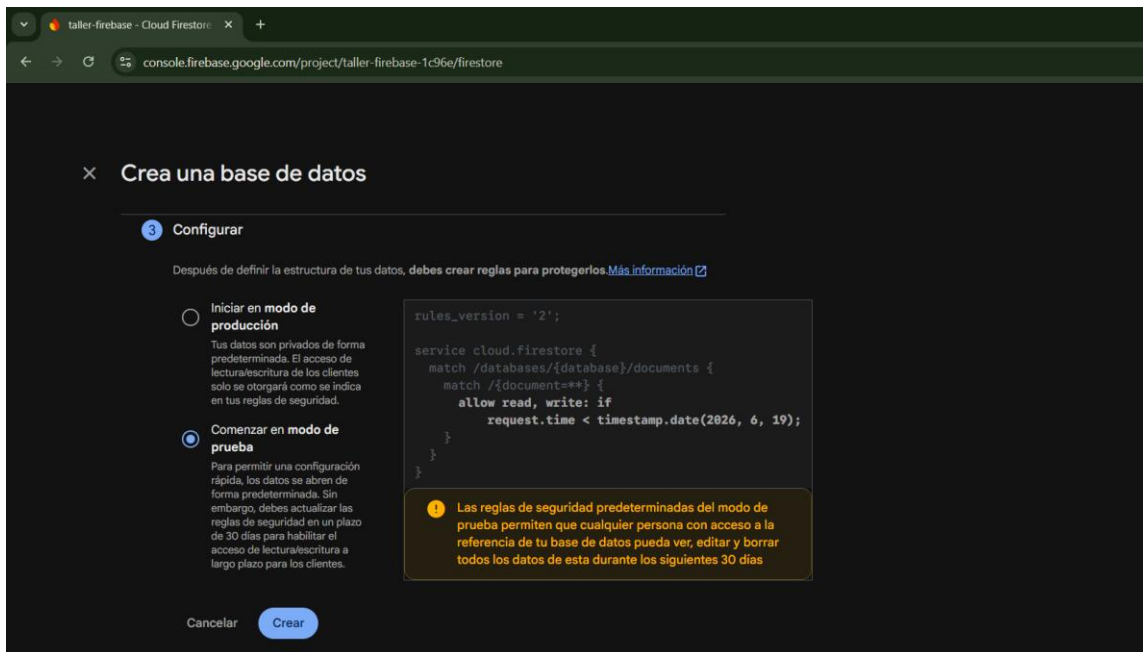
Ahora aquí es para crea la base de datos, dejamos seleccionada la opción estándar que es la gratuita le damos en siguiente.



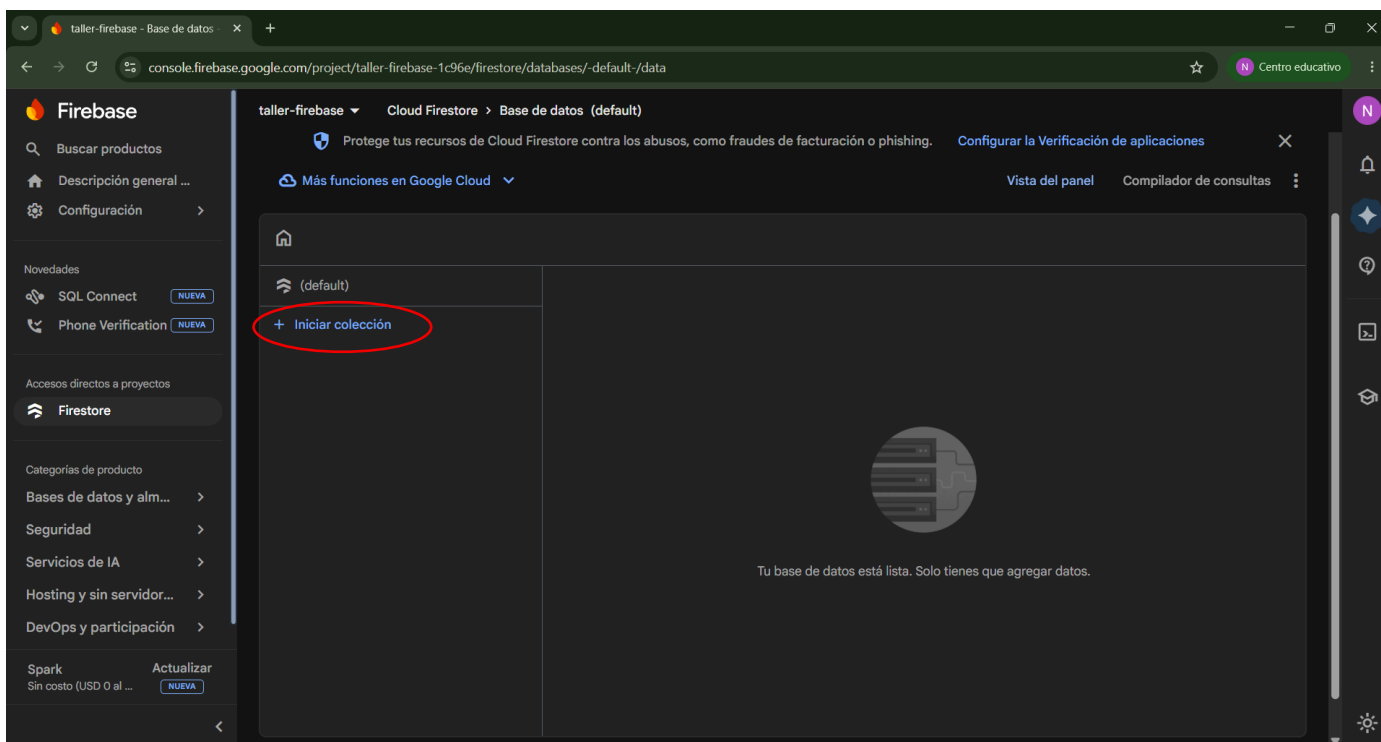
Después el paso siguiente nos dice que la base de datos obtendrá un id por default que se le asigna predeterminadamente, y que seleccionemos la ubicación que en este caso así se va a quedar todo y le damos click en siguiente.



Por último, Aquí seleccionamos en comenzar en modo de prueba ya que solo es una pequeña practica y en modo de producción se hacen mas configuraciones. Ya que seleccionamos en comenzar en modo de prueba le damos click en crear.



Ya cuando aparezca así, es que ya esta la base de datos creada en nuestro proyecto de Firebase. Vamos a agregar una “Colección o tabla” en la base de datos dándole click en Iniciar colección.



Después va a aparecer así, le asignamos el nombre/Id de tareas tal cual se muestra en la imagen porque vamos a utilizar este nombre en el código. Le damos en siguiente.

Inicia una colección

1 Asignar un ID a la colección — 2 Agregar el primer documento

Ruta superior
/

ID de la colección ?
tareas

Cancelar Siguiente

Ahora nos pide que agreguemos un documento que es como una tabla, ya que esta es una base de datos NoSQL. Llenamos los campos que se nos pide con la siguiente información y en la parte de abajo donde dice agregar campo le damos click para agregar otro más.

Inicia una colección

✓ Asignar un ID a la colección — 2 Agregar el primer documento

Ruta superior del documento
/tareas

ID de documento ?
tarea_1

Campo = Tipo
titulo = string

String
Estudiar Firebase

+ Agregar campo

Cancelar Guardar

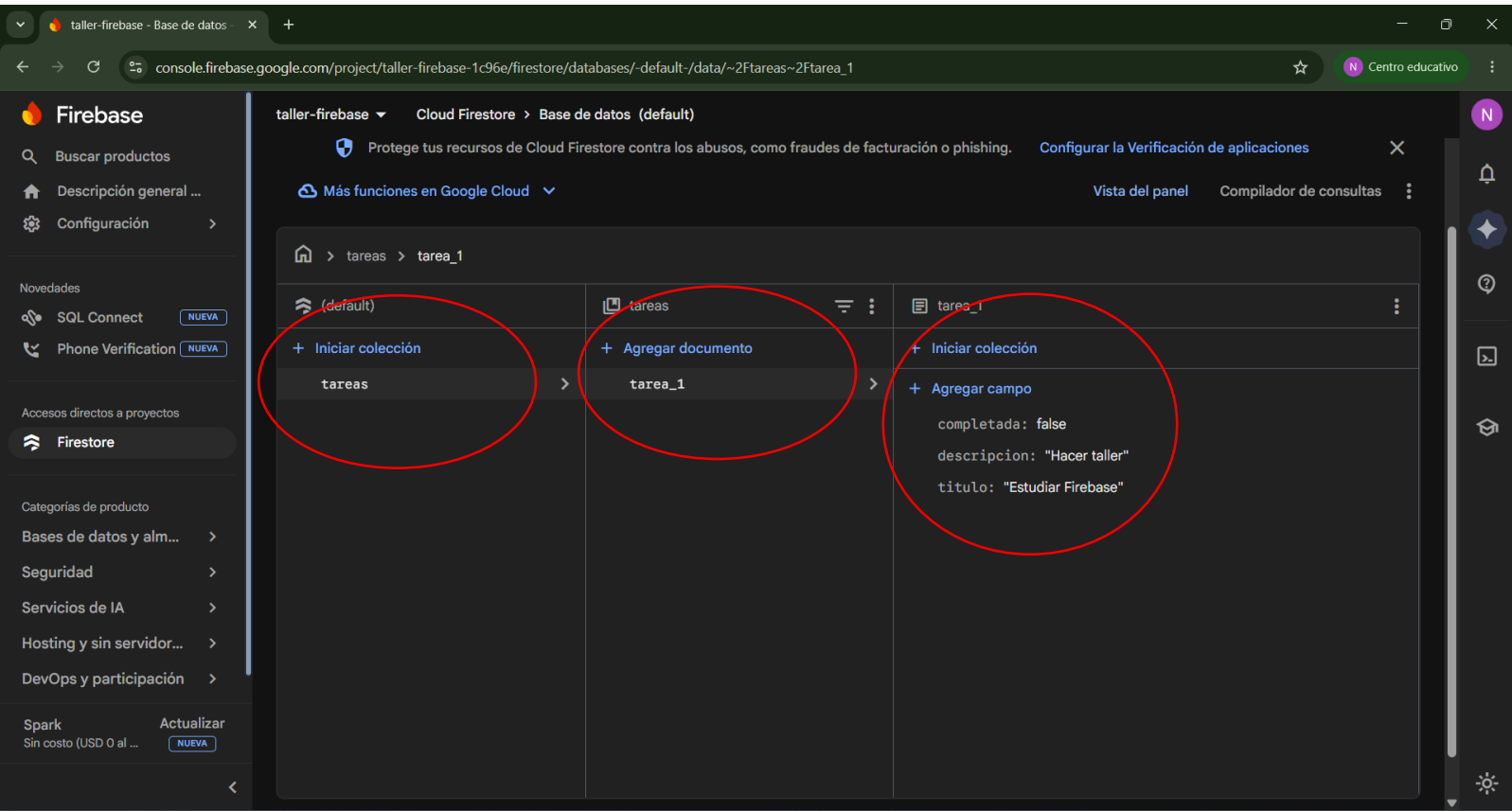
Aquí esta otro campo y agregamos la información a las cajas y le damos en agregar otro campo.

The screenshot shows a configuration interface with a dark background. At the top, there are two labels: 'Campo' and 'Tipo'. Below 'Campo' is a text input field containing 'descripcion'. To its right is an equals sign, followed by a dropdown menu showing 'string' and a minus icon. Below the 'descripcion' field, there is a label 'String' and a text input field containing 'Hacer taller'. At the bottom left, there is a plus icon and the text 'Agregar campo'. At the bottom right, there are two buttons: 'Cancelar' and 'Guardar'.

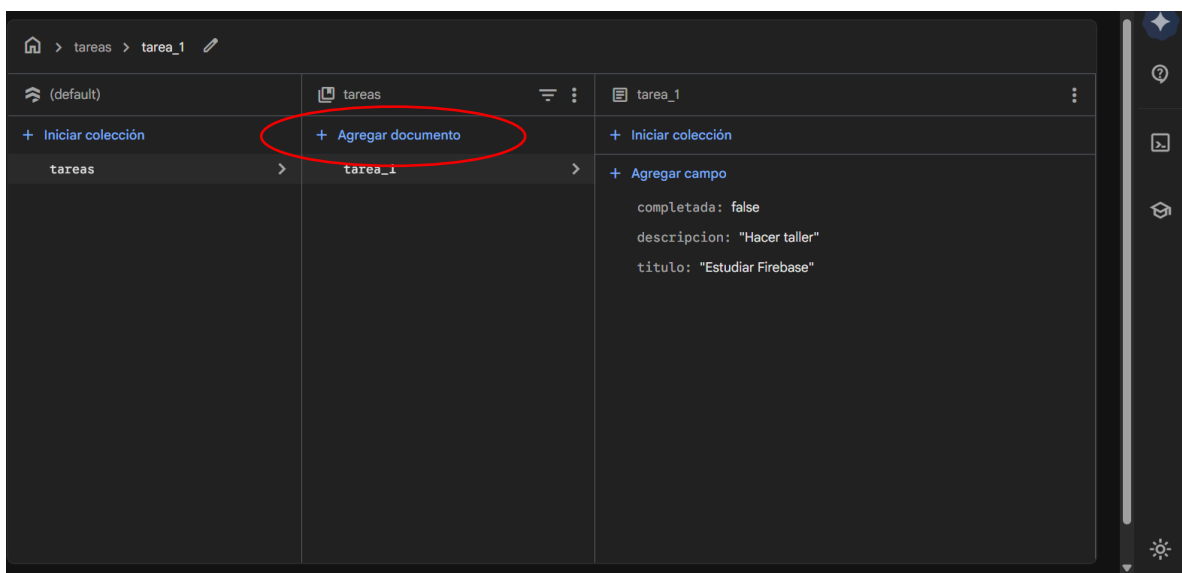
Aquí agregamos también la información que debe ir en este campo, y cuando ya hayamos terminado de agregar los campos y agregarles la información le damos click en guardar.

The screenshot shows a configuration interface with a dark background. At the top, there are three labels: 'Campo', 'Tipo', and 'Valor'. Below 'Campo' is a text input field containing 'completada'. To its right is an equals sign, followed by a dropdown menu showing 'boolean' and a minus icon. To the right of the 'boolean' dropdown is another dropdown menu showing 'falso' and a minus icon. Below the 'completada' field, there is a plus icon and the text 'Agregar campo'. At the bottom right, there are two buttons: 'Cancelar' and 'Guardar'.

Aquí va a aparecer la colección, los documentos que esta colección tiene y los campos que se agregaron.



Ahora donde dice agregar documento, que es en la parte de en medio. Le damos en agregar documento.



Aquí le agregamos un nombre al documento y agregamos los mismos campos anteriores, solo que ahora con otro resultado o contenido y ya cuando tengamos esto pues le damos click en guardar.

Cloud Firestore > Base de datos (default)

Agregar un documento

Ruta superior
/tareas

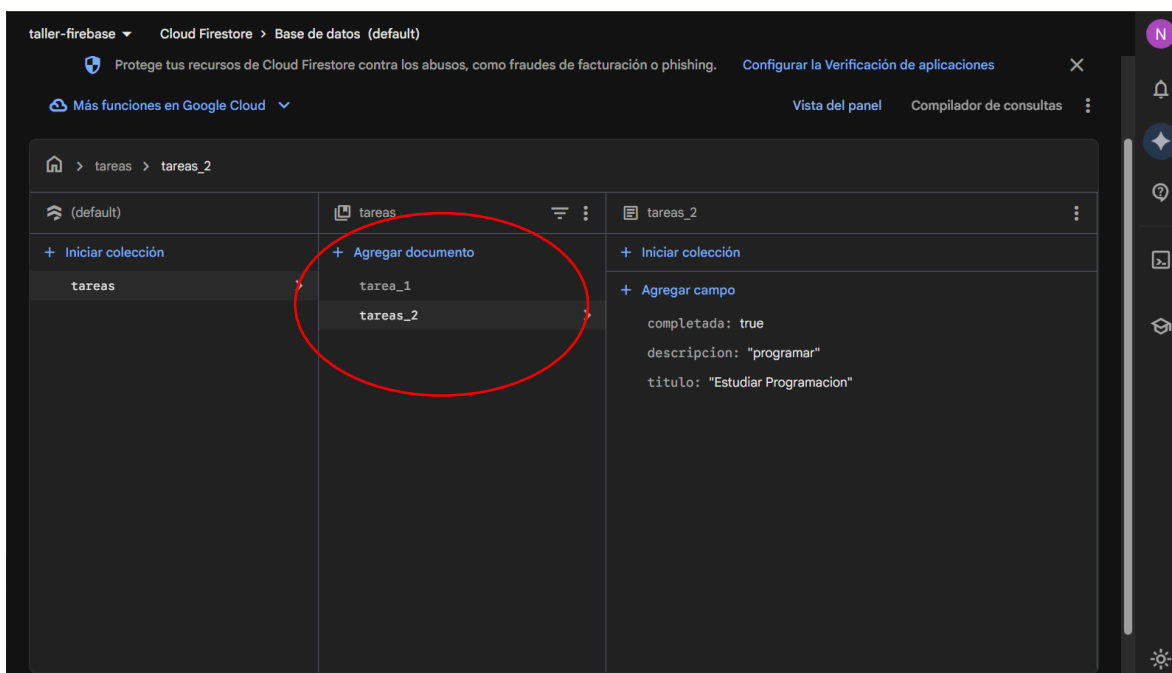
Campo	Tipo	
titulo	string	⊖
String		
Estudiar Programacion		

Campo	Tipo	
descripcion	string	⊖
String		
programar		

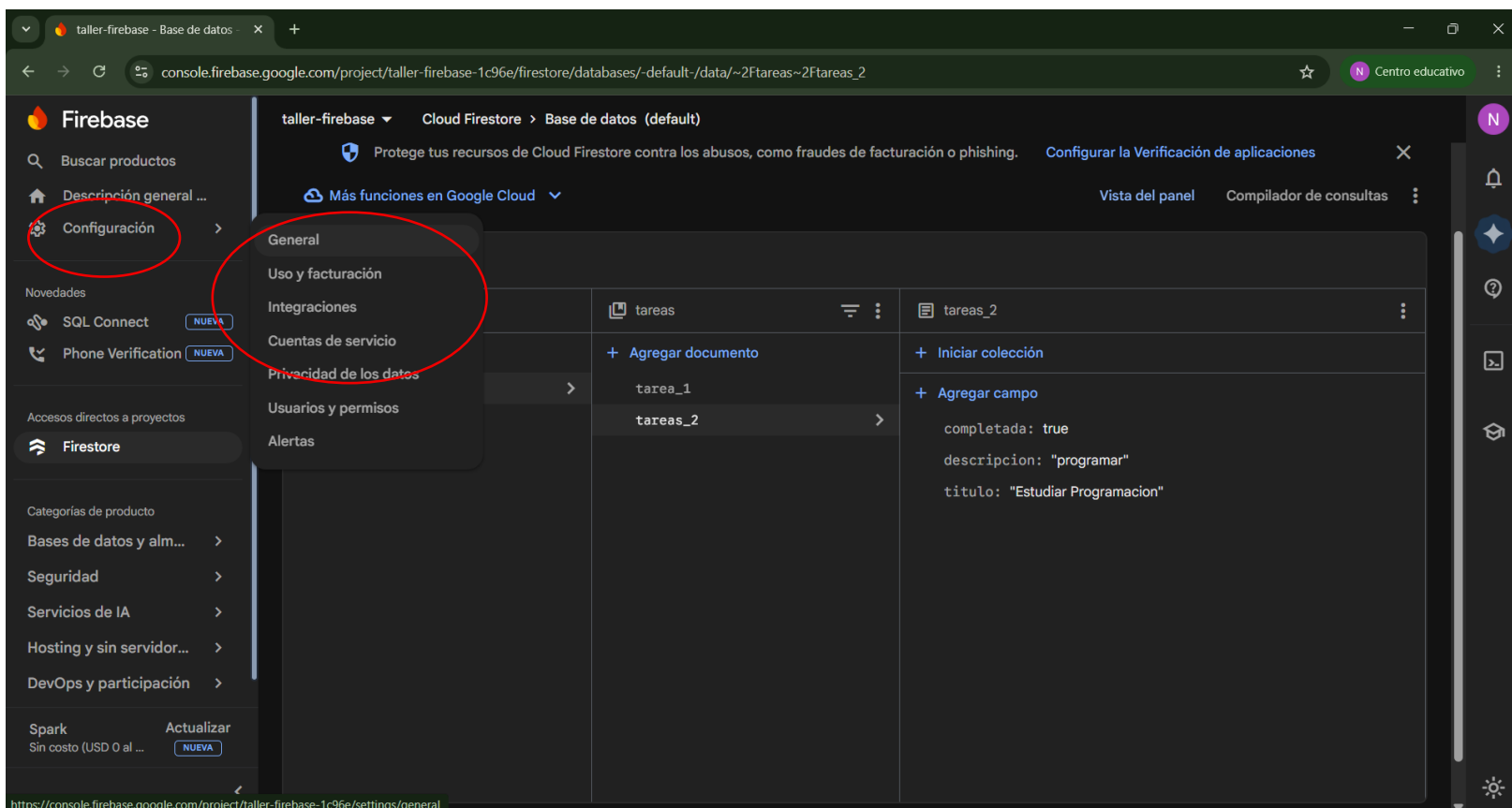
Campo	Tipo	Valor	
completada	boolean	verdadero	⊖

Cancelar **Guardar**

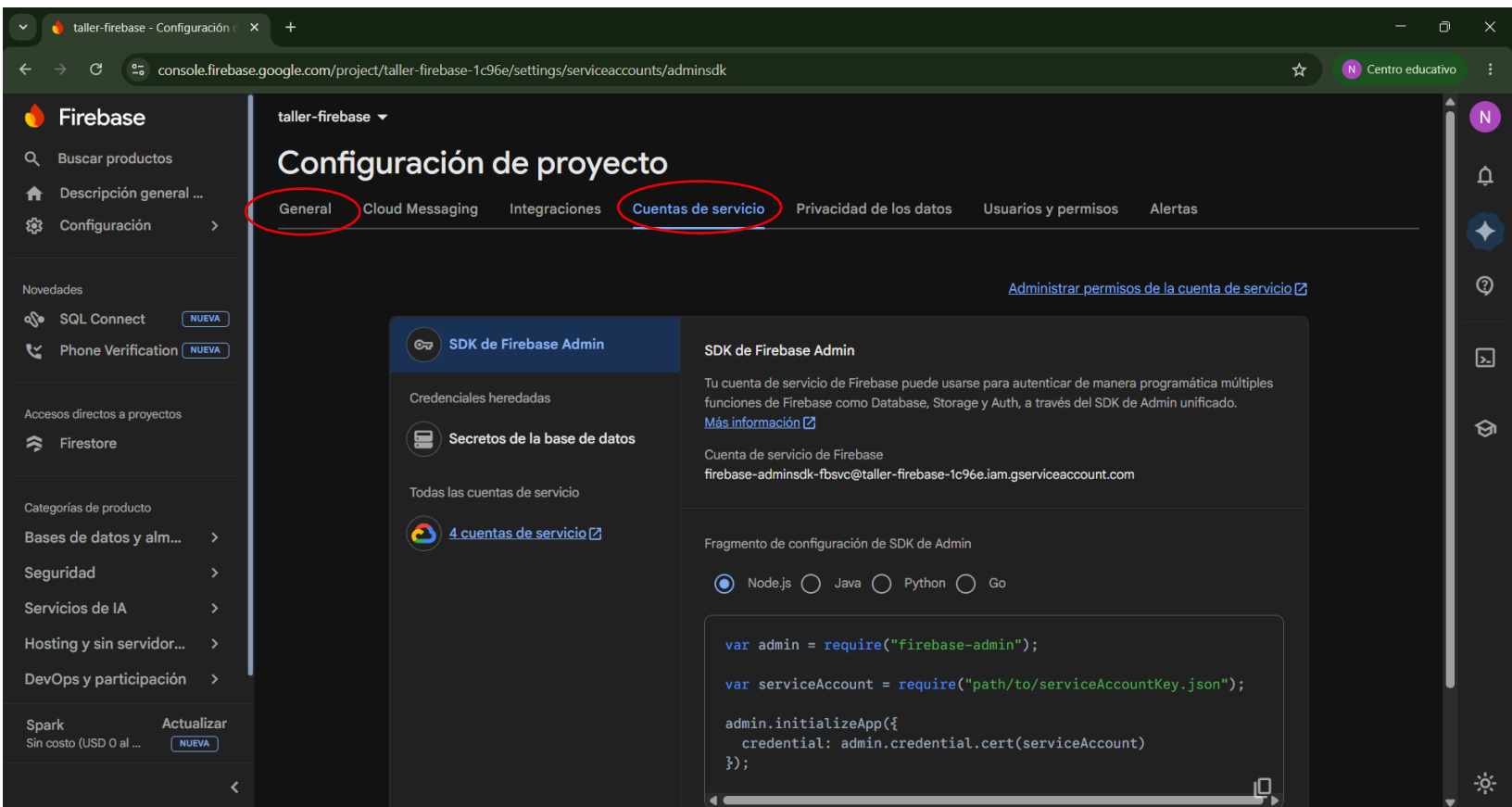
Aquí ya se puede ver que ya existen dos documentos en esta colección.



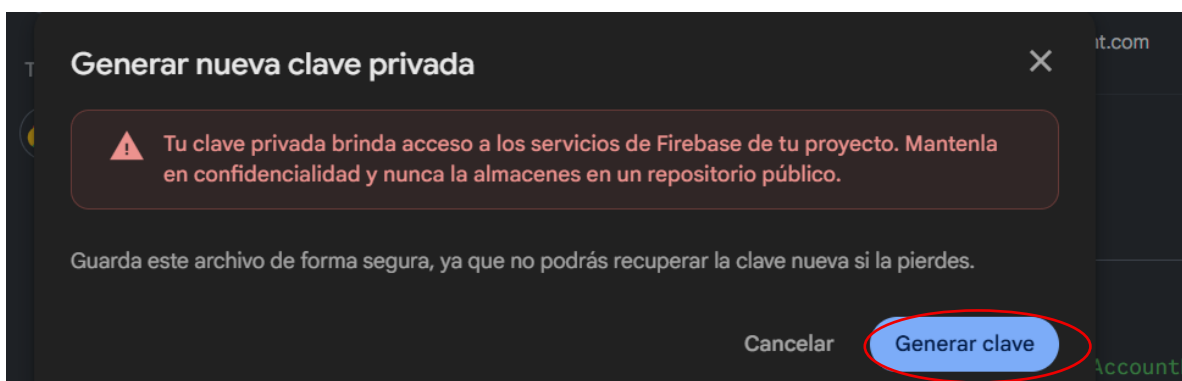
Ahora nos vamos al menú lateral izquierdo y buscamos la opción de configuración y ubicamos el mouse ahí y nos saldrá una ventanita con opciones y buscamos general y le damos click.



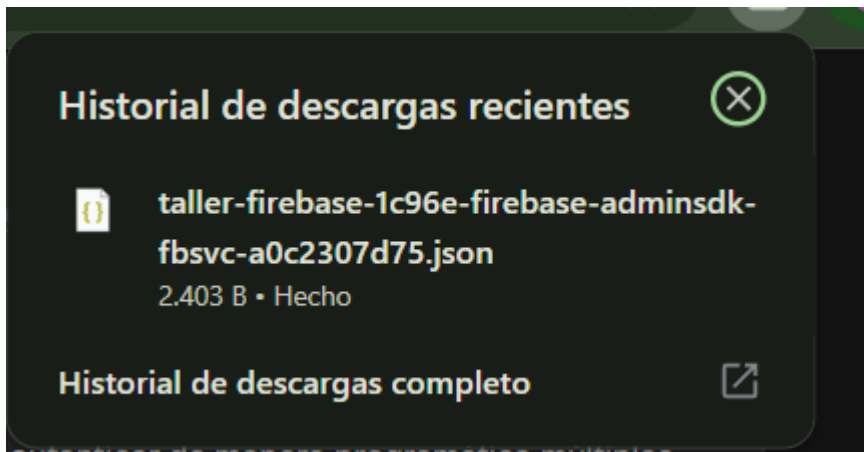
Ahora nos aparece así, bueno cuando le das click te va a aparecer en la pestaña general y aquí escogimos la de cuentas de servicio. Cuando deslizamos hasta debajo de este apartado, aparece un botón que dice generar clave privada le damos click.



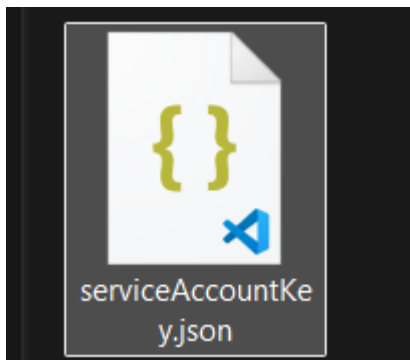
Aparecera esto, que indica que no podemos esa clave porque es privada y peligroso si alguien mas la utiliza. Le damos click en generar clave.



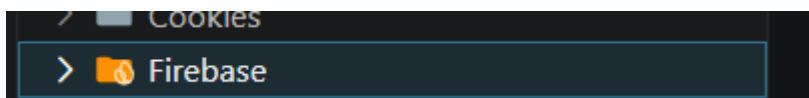
Se va a descargar un archivo json.



Vamos a nuestras descargas y le cambiamos el nombre al archivo para que sea más fácil agregar la ruta en el código al momento de usarlo.



Ahora nos vamos a el editor de texto que estamos utilizando y creamos la carpeta de nuestro taller.



Una vez creada la carpeta, ingresamos a la terminal entramos a la carpeta del proyecto, inicializamos el proyecto para que se creen los package.json.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  DEVD
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres> cd Firebase
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\Firebase> npm init -y
Wrote to C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\Firebase\package.json:

{
  "name": "firebase",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Ahora instalamos express y Firebase-admin que es lo que vamos a utilizar.

```
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\Firebase> npm install express firebase-admin
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead
npm warn deprecated uuid@8.3.2: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
added 229 packages, and audited 230 packages in 26s

45 packages are looking for funding
  run `npm fund` for details

8 low severity vulnerabilities

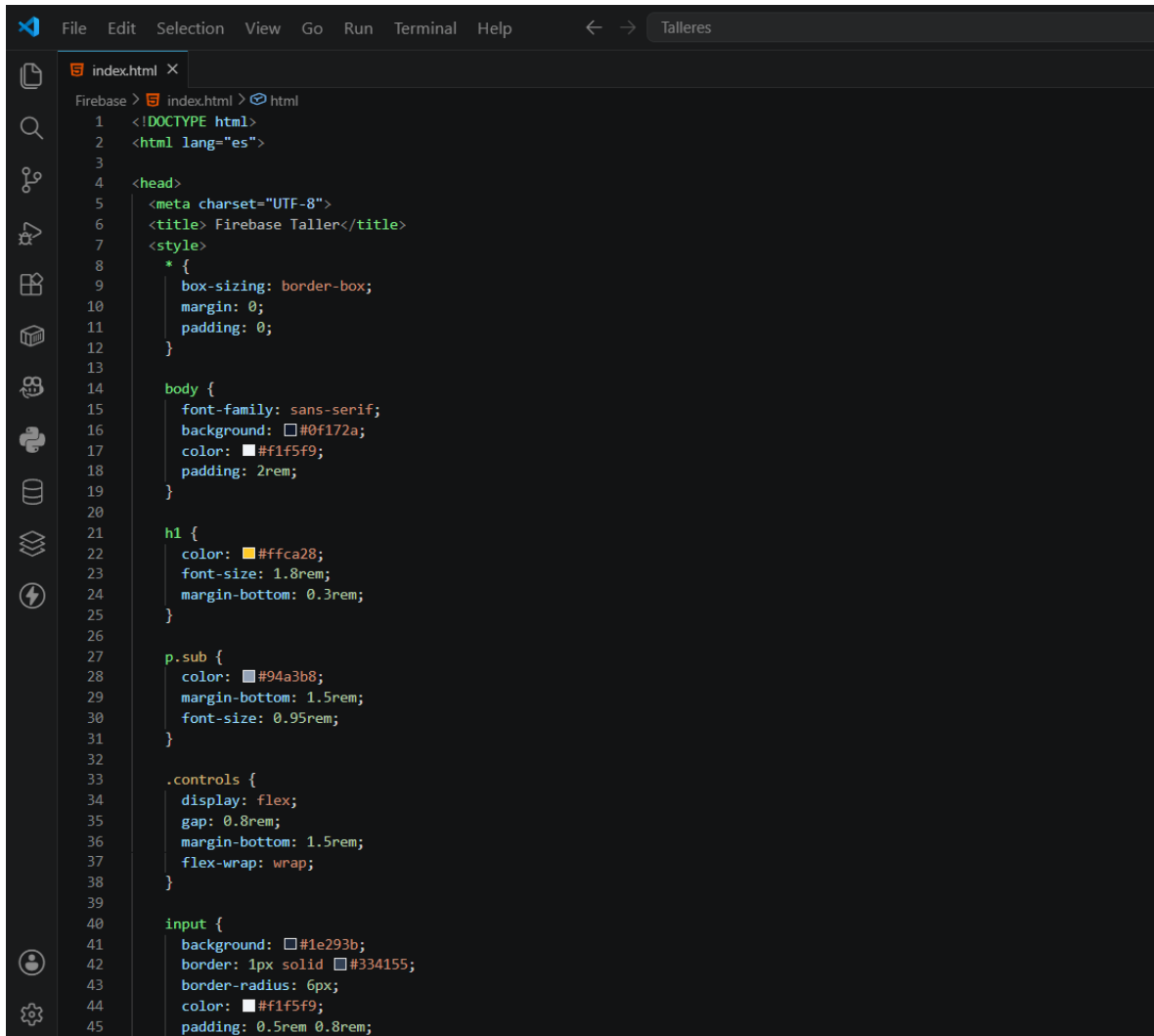
To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\Firebase>
```

Creamos un archivo llamado index.html en la carpeta raíz y le damos la estructura de html:5 y agregamos un título.

```
index.html X
Firebase > index.html > html > head > title
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Taller Firebase</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```

Luego de eso agregamos el código para el estilo de la pagina y los colores para que se vea agradable. Ya que es bastante código para el estilo.



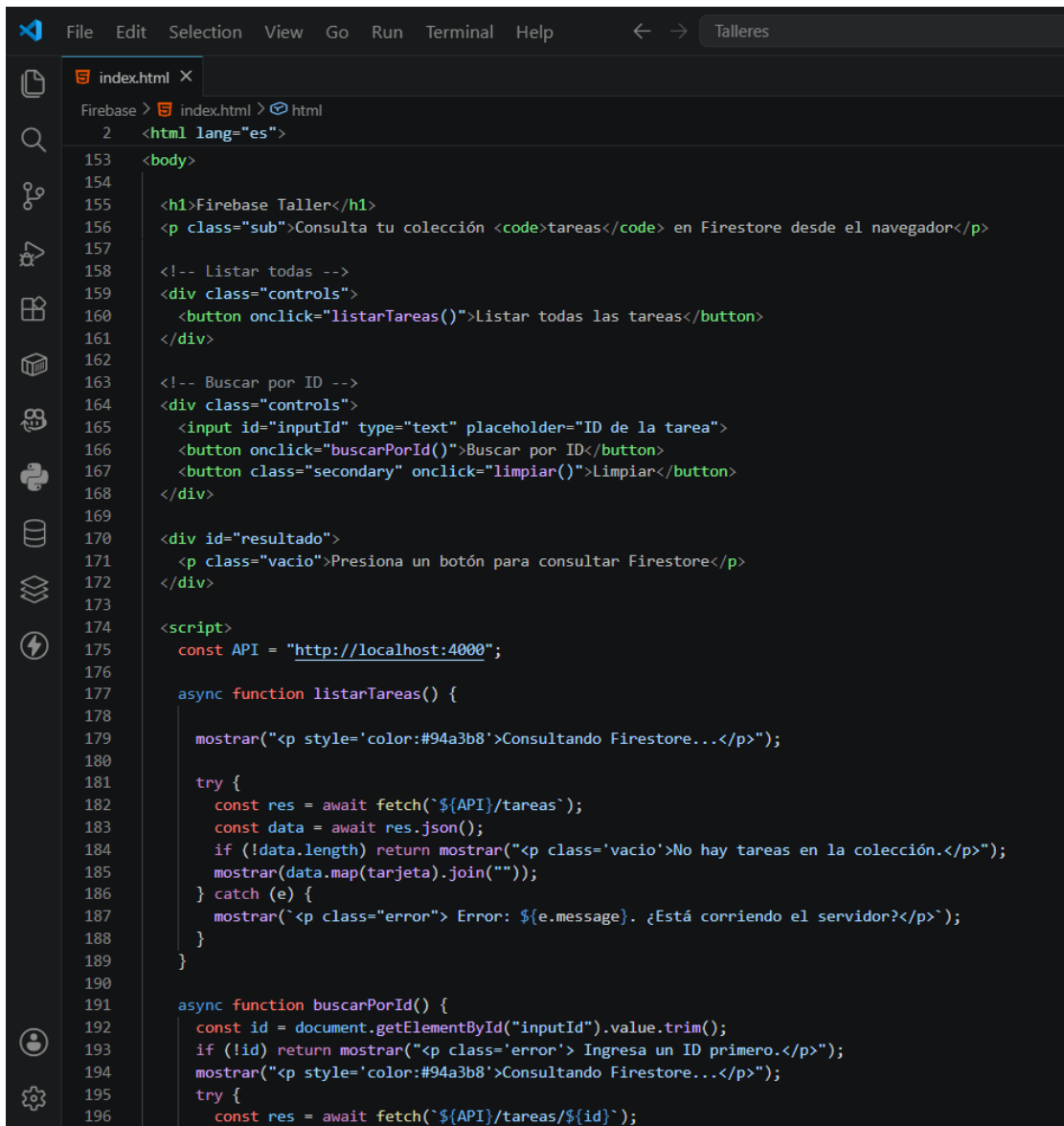
```
index.html X
Firebase > index.html > html
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5   <meta charset="UTF-8">
6   <title> Firebase Taller</title>
7   <style>
8     * {
9       box-sizing: border-box;
10      margin: 0;
11      padding: 0;
12    }
13
14    body {
15      font-family: sans-serif;
16      background: #0f172a;
17      color: #f1f5f9;
18      padding: 2rem;
19    }
20
21    h1 {
22      color: #ffca28;
23      font-size: 1.8rem;
24      margin-bottom: 0.3rem;
25    }
26
27    p.sub {
28      color: #94a3b8;
29      margin-bottom: 1.5rem;
30      font-size: 0.95rem;
31    }
32
33    .controls {
34      display: flex;
35      gap: 0.8rem;
36      margin-bottom: 1.5rem;
37      flex-wrap: wrap;
38    }
39
40    input {
41      background: #1e293b;
42      border: 1px solid #334155;
43      border-radius: 6px;
44      color: #f1f5f9;
45      padding: 0.5rem 0.8rem;
```

Este código crea una interfaz web conectada a Firebase para consultar tareas almacenadas en Firestore. Primero muestra una página HTML con botones y campos de texto para interactuar con la base de datos. La constante API guarda la dirección del servidor local (<http://localhost:4000>) al que se enviarán las peticiones.

La función `listarTareas()` se ejecuta cuando el usuario presiona el botón “Listar todas las tareas”. Esta función hace una petición `fetch` al endpoint `/tareas`, obtiene todas las tareas guardadas en Firestore y las muestra en pantalla. Si no existen tareas o ocurre un error, se muestra un mensaje correspondiente.

La función `buscarPorId()` permite buscar una tarea específica usando su ID. Primero toma el valor escrito en el campo de texto, verifica que no esté vacío y después envía una petición al servidor usando la ruta `/tareas/id`. Si la tarea existe, se muestra su información; si no, aparece un mensaje de error.

También existe una función `limpiar()` que borra el contenido mostrado en pantalla y reinicia la interfaz. En general, el código sirve como una pequeña aplicación web para consultar información almacenada en Firebase Firestore desde el navegador usando JavaScript y Express.

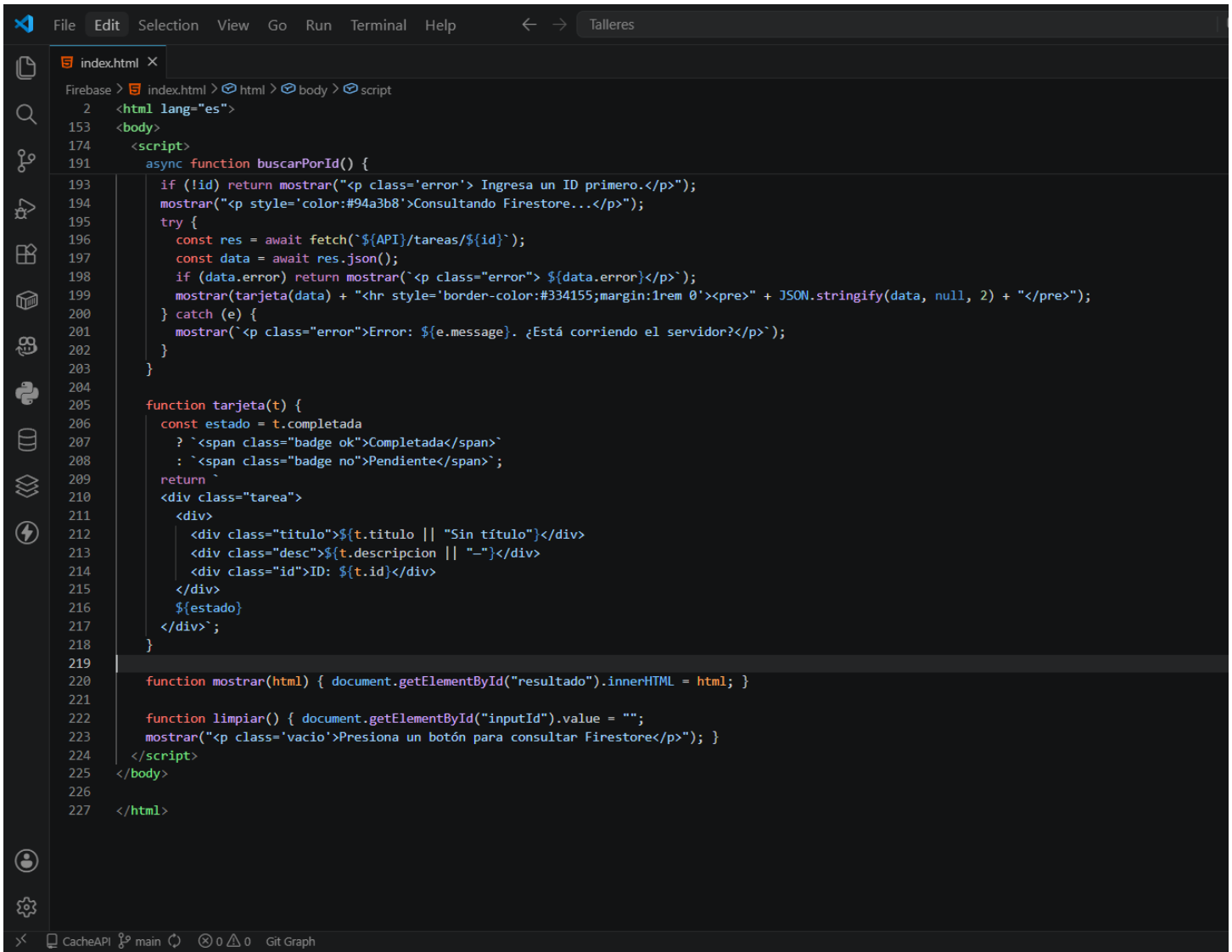


```
File Edit Selection View Go Run Terminal Help
index.html x
Firebase > index.html > html
2 <html lang="es">
153 <body>
154
155 <h1>Firebase Taller</h1>
156 <p class="sub">Consulta tu colección <code>tareas</code> en Firestore desde el navegador</p>
157
158 <!-- Listar todas -->
159 <div class="controls">
160 <button onclick="listarTareas()">Listar todas las tareas</button>
161 </div>
162
163 <!-- Buscar por ID -->
164 <div class="controls">
165 <input id="inputId" type="text" placeholder="ID de la tarea">
166 <button onclick="buscarPorId()">Buscar por ID</button>
167 <button class="secondary" onclick="limpiar()">Limpiar</button>
168 </div>
169
170 <div id="resultado">
171 <p class="vacio">Presiona un botón para consultar Firestore</p>
172 </div>
173
174 <script>
175 const API = "http://localhost:4000";
176
177 async function listarTareas() {
178
179   mostrar("<p style='color:#94a3b8'>Consultando Firestore...</p>");
180
181   try {
182     const res = await fetch(`${API}/tareas`);
183     const data = await res.json();
184     if (!data.length) return mostrar("<p class='vacio'>No hay tareas en la colección.</p>");
185     mostrar(data.map(tarjeta).join(""));
186   } catch (e) {
187     mostrar("<p class='error'> Error: ${e.message}. ¿Está corriendo el servidor?</p>");
188   }
189 }
190
191 async function buscarPorId() {
192   const id = document.getElementById("inputId").value.trim();
193   if (!id) return mostrar("<p class='error'> Ingresa un ID primero.</p>");
194   mostrar("<p style='color:#94a3b8'>Consultando Firestore...</p>");
195   try {
196     const res = await fetch(`${API}/tareas/${id}`);
```


Este fragmento del código continúa con la lógica de la aplicación web conectada a Firebase y se encarga de buscar tareas, mostrarlas visualmente y limpiar la interfaz. La función `buscarPorId()` realiza una consulta al servidor usando el ID escrito por el usuario. Primero verifica que el campo no esté vacío y después hace una petición con `fetch` a la ruta `/tareas/id`. Si la tarea existe, muestra sus datos en pantalla en formato JSON; si ocurre un error o el servidor no responde, se muestra un mensaje de error.

La función `tarjeta(t)` crea dinámicamente el diseño HTML de cada tarea para mostrarla de manera organizada. Presenta el título, descripción e ID de la tarea, además de indicar si está “Completada” o “Pendiente” dependiendo del valor de `t.completada`. Esto permite que las tareas aparezcan como tarjetas visuales dentro de la página.

La función `mostrar (HTML)` inserta contenido HTML dentro del elemento resultado, actualizando la información visible en la página. Finalmente, la función `limpiar()` borra el texto del campo de búsqueda y restablece el mensaje inicial de la interfaz. En conjunto, este código permite consultar, visualizar y administrar información obtenida desde Firestore de forma dinámica y sencilla en el navegador.



```
File Edit Selection View Go Run Terminal Help  ← →  Talleres

index.html X
Firebase > index.html > html > body > script
2  <html lang="es">
153 <body>
174   <script>
191     async function buscarPorId() {
193       if (!id) return mostrar("<p class='error'> Ingresa un ID primero.</p>");
194       mostrar("<p style='color:#94a3b8'>Consultando Firestore...</p>");
195       try {
196         const res = await fetch(`${API}/tareas/${id}`);
197         const data = await res.json();
198         if (data.error) return mostrar("<p class='error'> ${data.error}</p>");
199         mostrar(tarjeta(data) + "<hr style='border-color:#334155;margin:1rem 0'><pre>" + JSON.stringify(data, null, 2) + "</pre>");
200       } catch (e) {
201         mostrar("<p class='error'>Error: ${e.message}. ¿Está corriendo el servidor?</p>");
202       }
203     }
204
205     function tarjeta(t) {
206       const estado = t.completada
207         ? `<span class="badge ok">Completada</span>`
208         : `<span class="badge no">Pendiente</span>`;
209       return `
210 <div class="tarea">
211   <div>
212     <div class="titulo">${t.titulo || "Sin título"}</div>
213     <div class="desc">${t.descripcion || "-"}</div>
214     <div class="id">ID: ${t.id}</div>
215   </div>
216   ${estado}
217 </div>`;
218     }
219
220     function mostrar(html) { document.getElementById("resultado").innerHTML = html; }
221
222     function limpiar() { document.getElementById("inputId").value = "";
223       mostrar("<p class='vacio'>Presiona un botón para consultar Firestore</p>"); }
224   </script>
225 </body>
226
227 </html>
```

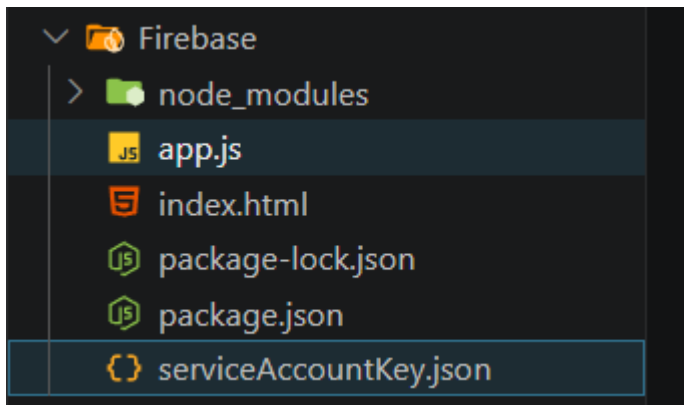
Este código corresponde a la configuración inicial del servidor con Firebase utilizando Node.js y Express. Primero se importan las librerías necesarias: express para crear el servidor web, firebase-admin para conectarse con Firebase desde el backend y path para manejar rutas de archivos.

Después se carga el archivo serviceAccountKey.json, el cual contiene las credenciales privadas del proyecto Firebase. Estas credenciales permiten autenticar y conectar el servidor con Firestore mediante admin.initializeApp(). Una vez realizada la conexión, se muestra en consola el identificador del proyecto para confirmar que Firebase fue inicializado correctamente.

Posteriormente se crea una instancia de Firestore con admin.firestore(), la cual servirá para realizar operaciones sobre la base de datos. También se inicializa la aplicación Express con express () y se define la constante COLECCION, que almacena el nombre de la colección llamada "tareas" dentro de Firestore. En conjunto, este fragmento prepara el entorno necesario para que la aplicación pueda comunicarse con Firebase y gestionar información almacenada en la nube.

```
JS app.js X
Firebase > JS app.js > ...
1  const express = require('express');
2  const admin = require('firebase-admin');
3  const path = require('path');
4
5  // 1.- Inicializar Firebase Admin
6  const serviceAccount = require("./serviceAccountKey.json");
7
8  admin.initializeApp({
9    credential:admin.credential.cert(serviceAccount),
10
11  });
12
13  console.log("Proyecto conectado:", admin.app().options.credential.projectId);
14
15  const db = admin.firestore();
16  const app = express();
17  const COLECCION = "tareas";
```

Aquí en la carpeta del proyecto, agregamos el archivo que descargamos cuando generamos clave privada. Para que funcione ya que anteriormente agregamos la ruta y para que funcione ya correctamente agregamos el archivo a la carpeta.



El código mostrado es una aplicación en Node.js con Express y Firebase Firestore que maneja tareas. Primero define la colección llamada "tareas" en la base de datos. Después crea una ruta principal ("/") que envía y muestra el archivo index.html al navegador usando sendFile.

También define una ruta GET llamada "/tareas" que obtiene todas las tareas almacenadas en Firestore de forma asíncrona. Dentro de un bloque try, consulta la colección "tareas", recorre los documentos obtenidos y crea un arreglo con el id y los datos de cada tarea. Finalmente, responde en formato JSON con la lista de tareas.

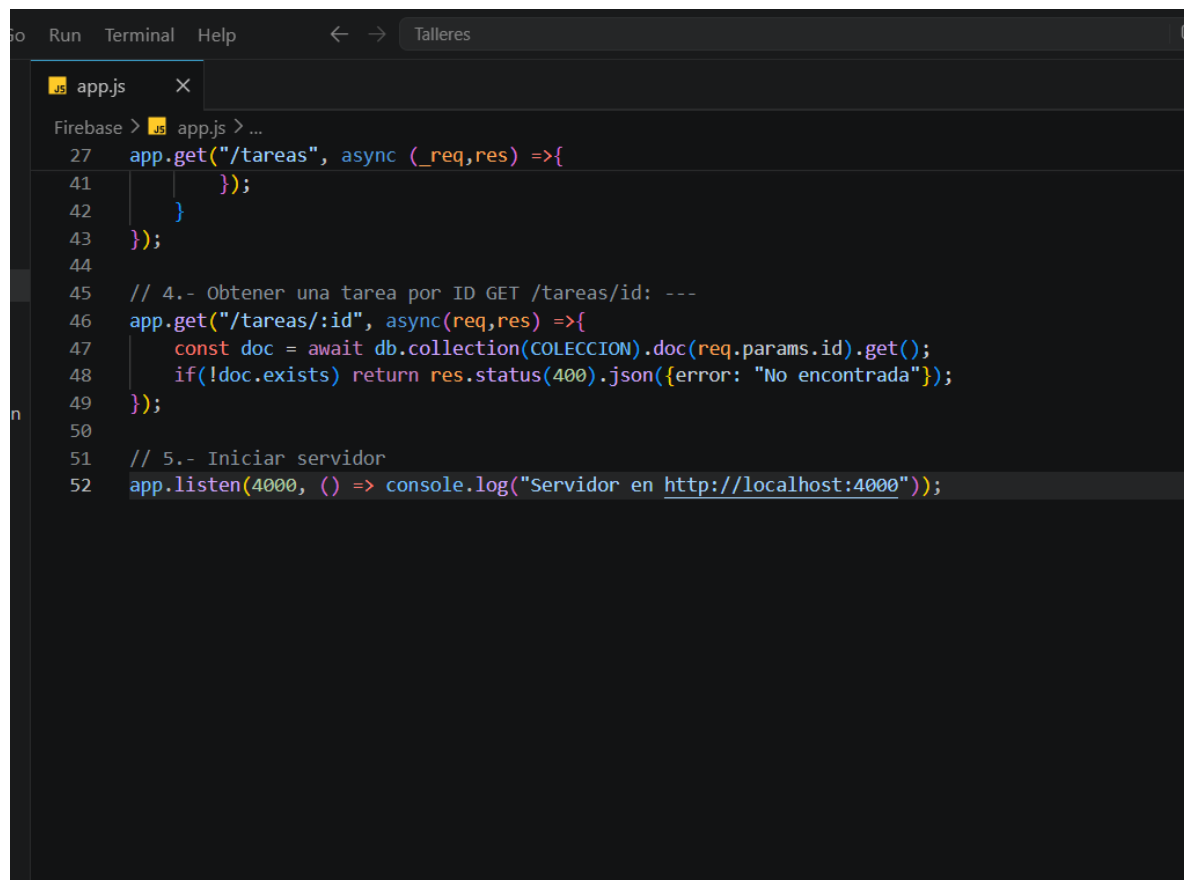
Si ocurre algún error durante la consulta, el bloque catch captura la excepción, la muestra en consola y devuelve una respuesta con código 500 junto con el mensaje del error en formato JSON.

```
Run Terminal Help  < -> Talleres
app.js x
Firebase > us app.js > ...
17  const COLECCION = "tareas";
18
19  // 2.- Servir la pagina HTML
20  app.get("/", (_req, res) =>{
21    res.sendFile(path.join(__dirname, "index.html"));
22  });
23
24
25  // 3.- Listar todas las tareas GET/tareas
26  app.get("/tareas", async (_req,res) =>{
27    try{
28      const app = await db.collection("tareas").get();
29      const tareas = snap.docs.map(doc =>({
30        id: doc.id,
31        ...doc.data()
32      }));
33
34      res.json(tareas);
35    }catch(error){
36      console.error("ERROR:");
37      console.error(error);
38      res.status(500).json({
39        error: error.message
40      });
41    }
42  });
43
```

El código mostrado agrega una nueva ruta GET llamada `"/tareas/:id"` en una aplicación Node.js con Express y Firebase Firestore. Esta ruta permite obtener una tarea específica mediante su identificador (id).

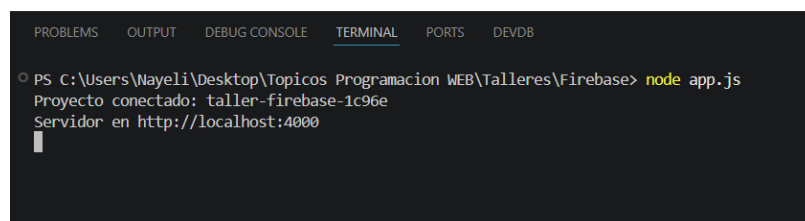
Cuando el usuario envía una petición, el código busca en la colección definida (COLECCION) el documento cuyo ID coincide con `req.params.id`. Después verifica si el documento existe; si no existe, devuelve una respuesta con código 400 y un mensaje JSON indicando "No encontrada".

Finalmente, el código inicia el servidor usando `app.listen(4000)`, dejando la aplicación funcionando en el puerto 4000 y mostrando en consola el mensaje Servidor en `http://localhost:4000`.



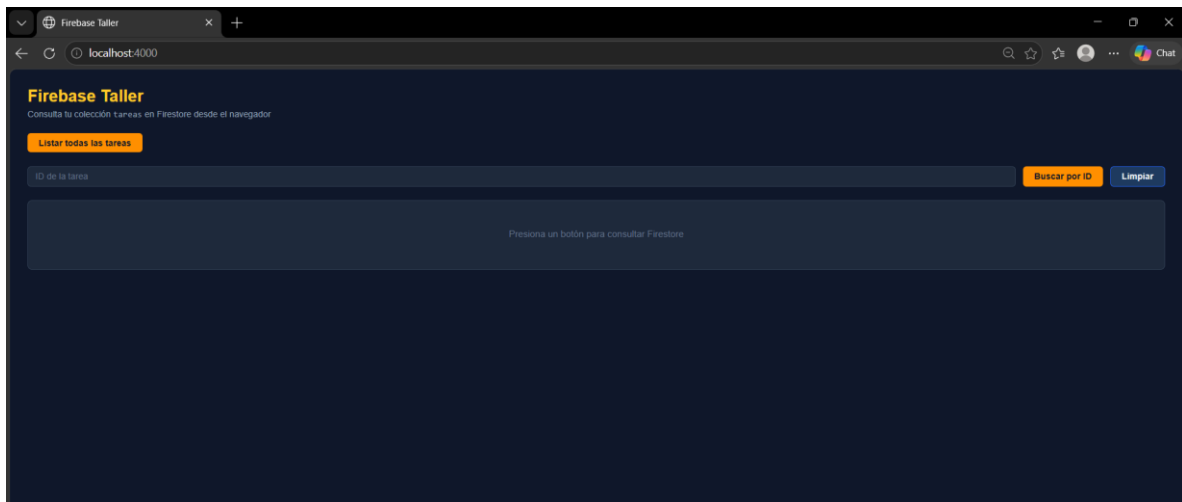
```
27 app.get("/tareas", async (_req,res) =>{
41   });
42 }
43 });
44
45 // 4.- Obtener una tarea por ID GET /tareas/id: ---
46 app.get("/tareas/:id", async(req,res) =>{
47   const doc = await db.collection(COLECCION).doc(req.params.id).get();
48   if(!doc.exists) return res.status(400).json({error: "No encontrada"});
49 });
50
51 // 5.- Iniciar servidor
52 app.listen(4000, () => console.log("Servidor en http://localhost:4000"));
```

Ya teniendo el index y el app listos con todo su código en la terminal que tenemos o abrimos y nos aseguramos que estemos dentro de la carpeta del proyecto y corremos la app con la instrucción `node app.js`.

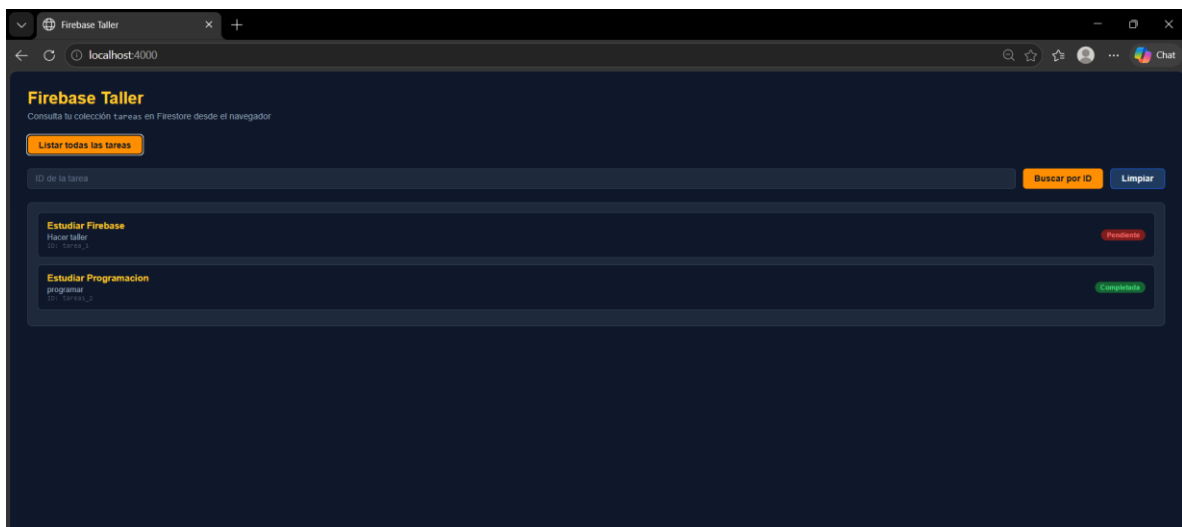


```
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\Firebase> node app.js
Proyecto conectado: taller-firebase-1c96e
Servidor en http://localhost:4000
```

Abrimos el localhost:4000 y nos aparece esto, que es la pequeña pagina que creamos, no nos dio error esperemos y funcione correctamente.



Vamos a darle click en donde dice Listar todas las tareas. Y tiene que mostrar las tareas o documentos que creamos al inicio en Firebase.



Pues como observamos nos muestra las tareas con su descripción, su nombre de tarea y su id. Ahí una barra de búsqueda para poder buscar por id y un botón de limpiar que si le damos click se borran las tareas, pero nada más de la interfaz. También podemos observar que sale si la tarea esta pendiente o esta completada ya que ese dato lo pusimos tipo boolean por eso aparece así. Es todo lo de la práctica ya vimos que funcionó correctamente.

Conclusion

En esta práctica se logró conocer e implementar las principales herramientas de Firebase para el desarrollo de aplicaciones web conectadas a la nube. A través de la creación de un proyecto, la configuración de Firestore y la conexión con Node.js y Express, fue posible comprender el funcionamiento de una base de datos NoSQL y la forma en que se pueden almacenar y consultar datos en tiempo real.

Además, se desarrolló una pequeña aplicación web capaz de listar y buscar tareas mediante peticiones al servidor, comprobando así la correcta comunicación entre Firebase Firestore, el backend y la interfaz web. También se aprendió a utilizar credenciales privadas para autenticar la aplicación y permitir el acceso seguro a la base de datos.

Los resultados obtenidos fueron satisfactorios, ya que la aplicación funcionó correctamente al mostrar la información almacenada en Firebase y permitir consultas dinámicas desde el navegador. En conclusión, Firebase es una plataforma muy útil y eficiente para el desarrollo moderno de aplicaciones, debido a que simplifica procesos, reduce tiempos de implementación y facilita la administración de datos en la nube de manera rápida y segura.